1. SALN Team 1

1.1. Setup and Deployment Guide

1.1.1. Members:

Batistil, Mansur; 202315263
Camacho, Reuter Jan; 202401096
Filio, Ethan; 202303295
Mislang, Gabriel; 202305587
Rodrigo, Dom; 202315079

2. ~~Nonmicroservices Architecture (Laravel's Default MVC)~~

~~***Deprecated Repo:*** https://github.com/chry607/SALN_Backend~~

~~The original application was a monolithic Laravel MVC app. It has since been deprecated in favour of the microservices architecture described below.~~

3. Microservices Architecture (Digital Ocean Droplet)

Active Repo: https://github.com/meezlung/saln-microservices/ (branch: **`oop`**)

The Digital Ocean deployment runs all three Laravel services as `php artisan serve` processes behind an nginx reverse proxy on a single Ubuntu 24.04 LTS droplet at **`saln1.upcsweb.dev`**.

3.1. Prerequisites

Install the following on the Ubuntu 24.04 LTS droplet:

```
sudo apt update
sudo apt install -y nginx git curl unzip
```

PHP 8.5 (via php.new / Herd Lite):

```
/bin/bash -c "$(curl -fsSL https://php.new/install/linux/8.5)"
```

Or install PHP 8.4 via `ondrej/php` PPA:

```
sudo add-apt-repository ppa:ondrej/php -y
sudo apt update
sudo apt install -y php8.4 php8.4-cli php8.4-fpm php8.4-mbstring php8.4-xml \
    php8.4-bcmath php8.4-pgsql php8.4-curl php8.4-zip php8.4-sqlite3
```

Composer 2:

```
curl -sS https://getcomposer.org/installer | php
sudo mv composer.phar /usr/local/bin/composer
```

Node.js v18.19.1:

```
curl -fsSL https://deb.nodesource.com/setup_18.x | sudo -E bash -
sudo apt install -y nodejs
```

pdftk (document service PDF generation):

```
sudo apt install -y pdftk
```

PostgreSQL client:

```
sudo apt install -y postgresql-client
```

Exact versions used in production:

Tool	Version
`php`	`8.5.0` (Herd Lite / php.new musl build)
`composer`	`2.8.12`
`node`	`v18.19.1`
`npm`	bundled with Node 18
`git`	`2.43.0`
`nginx`	`1.24.0 (Ubuntu)`
`psql`	`18.1`
`terraform`	`1.15.4`

3.2. Clone the Repository

```
git clone https://github.com/meezlung/saln-microservices.git
cd saln-microservices
# Digital Ocean setup uses the oop branch
git checkout oop
```

3.3. Database Setup

Connect to your PostgreSQL server and create the three databases before running migrations:

```
psql -h <db-host> -U postgres -p 5432
```

```
CREATE DATABASE saln_auth_db;
CREATE DATABASE saln_form_db;
CREATE DATABASE saln_document_db;
\q
```

3.4. Auth Service

```
cd services/auth-service
composer install
cp .env.example .env
# Edit .env: set DB_HOST, DB_PORT, DB_DATABASE=saln_auth_db,
#   DB_USERNAME, DB_PASSWORD, RESEND_API_KEY,
#   MAIL_FROM_ADDRESS=onboarding@upcsweb.dev, INTERNAL_SERVICE_TOKEN
php artisan key:generate
php artisan migrate
php artisan serve --host=127.0.0.1 --port=8001
```

3.5. Form Service

```
cd services/form-service
composer install
cp .env.example .env
# Edit .env: set DB_DATABASE=saln_form_db,
#   AUTH_SERVICE_URL=http://127.0.0.1:8001, INTERNAL_SERVICE_TOKEN
php artisan key:generate
php artisan migrate
php artisan serve --host=127.0.0.1 --port=8002
```

3.6. Document Service

```
cd services/document-service
composer install
cp .env.example .env
# Edit .env: set DB_DATABASE=saln_document_db,
#   AUTH_SERVICE_URL=http://127.0.0.1:8001, INTERNAL_SERVICE_TOKEN
php artisan key:generate
php artisan migrate
php artisan serve --host=127.0.0.1 --port=8003
```

Open a **separate terminal** to process queued PDF generation jobs:

```
cd services/document-service
php artisan queue:work
```

3.7. Frontend

```
cd frontend/web
npm install
npm run dev   # Vite dev server at http://127.0.0.1:5173
```

3.8. Quick Start with run-backend.sh

From the repo root, a convenience script starts all four processes and cleans up on Ctrl-C:

```
./run-backend.sh
```

Services started:

Service	Address
auth-service	`http://127.0.0.1:8001`
form-service	`http://127.0.0.1:8002`
document-service	`http://127.0.0.1:8003`
frontend (Vite)	`http://127.0.0.1:5173`

Note: `run-backend.sh` does not start `php artisan queue:work` for the document service. Start that separately if PDF generation is needed.

The team used **WezTerm** for terminal multiplexing to run all panes in one window. `screen` is a lightweight alternative:

```
screen -S saln   # new session
# Ctrl-A c = new window, Ctrl-A n = next window
```

3.9. Terminal Multiplexing

3.10. Useful Development Commands

```
php artisan config:clear
php artisan optimize:clear
php artisan cache:clear
```

3.11. nginx Configuration

The droplet runs nginx 1.24.0 as a reverse proxy. The site config is placed at `/etc/nginx/sites-enabled/saln-microservices`:

```
##
# You should look at the following URL's in order to grasp a solid understanding
# of Nginx configuration files in order to fully unleash the power of Nginx.
# https://www.nginx.com/resources/wiki/start/
# https://www.nginx.com/resources/wiki/start/topics/tutorials/config_pitfalls/
# https://wiki.debian.org/Nginx/DirectoryStructure
#
# In most cases, administrators will remove this file from sites-enabled/ and
# leave it as reference inside of sites-available where it will continue to be
# updated by the nginx packaging team.
#
# This file will automatically load configuration files provided by other
# applications, such as Drupal or Wordpress. These applications will be made
# available underneath a path with that package name, such as /drupal8.
```

```
#
# Please see /usr/share/doc/nginx-doc/examples/ for more detailed examples.
##

# Default server configuration
#
server {
        listen 80 default_server;
        listen [::]:80 default_server;

        # SSL configuration
        #
        # listen 443 ssl default_server;
        # listen [::]:443 ssl default_server;
        #
        # Note: You should disable gzip for SSL traffic.
        # See: https://bugs.debian.org/773332
        #
        # Read up on ssl_ciphers to ensure a secure configuration.
        # See: https://bugs.debian.org/765782
        #
        # Self signed certs generated by the ssl-cert package
        # Don't use them in a production server!
        #
        # include snippets/snakeoil.conf;

        # root /var/www/html;
        # root /var/www/saln/public;

        # React production build output
    root /var/www/saln-microservices/frontend/web/dist;

        # Add index.php to the list if you are using PHP
        index index.html;

        server_name 202.92.129.45;

    # ----- Frontend (React SPA) -----
    location / {
        try_files $uri $uri/ /index.html;
    }

    # Cache static assets
    location ~* \.(js|mjs|css|png|jpg|jpeg|gif|svg|ico|webp|woff|woff2|ttf)$ {
        expires 30d;
        add_header Cache-Control "public, max-age=2592000, immutable";
        try_files $uri =404;
    }

        # ----- Auth service -----
    # Example: /api/auth/login -> auth-service
        location /api/auth/ {
                rewrite ^/api/auth/?(.*)$ /api/$1 break;
                proxy_pass http://127.0.0.1:8001;
                proxy_http_version 1.1;
```

```
                proxy_set_header Host $host;
                proxy_set_header X-Real-IP $remote_addr;
                proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
                proxy_set_header X-Forwarded-Proto $scheme;
        }

    # Needed if Sanctum CSRF endpoint is used directly
    location /sanctum/ {
        proxy_pass http://127.0.0.1:8001/sanctum/;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    # ----- Form service -----
    # Example: /api/forms/... -> form-service
        location /api/forms/ {
                rewrite ^/api/forms/?(.*)$ /api/$1 break;
                proxy_pass http://127.0.0.1:8002;
                proxy_http_version 1.1;
                proxy_set_header Host $host;
                proxy_set_header X-Real-IP $remote_addr;
                proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
                proxy_set_header X-Forwarded-Proto $scheme;
        }

    # ----- Document service -----
    # Example: /api/documents/... -> document-service
        location /api/documents/ {
                #rewrite ^/api/documents/?(.*)$ /api/$1 break;
                root /var/www/saln-microservices/services/document-service/
public;
                proxy_pass http://127.0.0.1:8003;
                proxy_http_version 1.1;
                proxy_set_header Host $host;
                proxy_set_header X-Real-IP $remote_addr;
                proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
                proxy_set_header X-Forwarded-Proto $scheme;
        }
}


# Virtual Host configuration for example.com
#
# You can move that to a different file under sites-available/ and symlink that
# to sites-enabled/ to enable it.
#
#server {
#       listen 80;
#       listen [::]:80;
#
#       server_name example.com;
#
```

```
#       root /var/www/example.com;
#       index index.html;
#
#       location / {
#               try_files $uri $uri/ =404;
#       }
#}
```

After saving, reload nginx:

```
sudo nginx -t        # verify config syntax
sudo systemctl reload nginx
```

4. Microservices Architecture (AWS Lambda Function)

Active Repo: https://github.com/meezlung/saln-microservices/ (branch: **`aws-try`**)

The AWS deployment runs auth and form services as PHP Lambda functions (Bref runtime) behind a CloudFront distribution at **`saln1a.upcsweb.dev`**. The document service Lambda is managed separately by a collaborator.

4.1. Switch Branch

```
git checkout aws-try
```

4.2. Prerequisites

Tool	Version	Install
AWS CLI	`2.34.50`	`https://aws.amazon.com/cli/`
Serverless Framework	`4.36.1`	`npm install -g serverless`
Node.js	`v18.19.1`	NodeSource / nvm
Terraform	`1.15.4`	`https://developer.hashicorp.com/terraform`
Composer	`2.8.12`	see Digital Ocean section

Configure AWS credentials:

```
aws configure
# Enter: Access Key ID, Secret Access Key, region (ap-southeast-2), output (json)
```

4.3. Step 1 — Provision Aurora PostgreSQL Serverless v2

1. AWS Console → **RDS** → **Create database** → **Aurora** → **PostgreSQL-compatible** → **Serverless v2**
2. Region: **`us-east-1`** (the cluster must be publicly accessible for cross-region Lambda access)
3. Enable **Publicly accessible**

4. Add an inbound security group rule: **TCP port 5432** from `0.0.0.0/0`

Connect via psql and create the three databases:

```
psql -h <cluster-endpoint> -U postgres -p 5432 --set=sslmode=require
```

```
CREATE DATABASE saln_auth_db;
CREATE DATABASE saln_form_db;
CREATE DATABASE saln_document_db;
\q
```

4.4. Step 2 — Create S3 Bucket for PDFs

AWS Console → **S3** → **Create bucket**

- Name: `saln-documents-pdfs`
- Region: `ap-southeast-2`
- Block all public access (enabled)

4.5. Step 3 — Deploy Auth Service

```
cd services/auth-service
composer install --no-dev --optimize-autoloader --ignore-platform-reqs
# serverless.yml already contains all env vars
sudo serverless deploy
```

Note the **Function URL** printed after deployment. Then run migrations:

```
serverless bref:cli --args="migrate --force"
```

4.6. Step 4 — Deploy Form Service

```
cd services/form-service
composer install --no-dev --optimize-autoloader --ignore-platform-reqs
# Ensure AUTH_SERVICE_URL in serverless.yml = auth Lambda URL from Step 3
sudo serverless deploy
serverless bref:cli --args="migrate --force"
```

4.7. Step 5 — Document Service

The document service Lambda is managed separately by a collaborator and is already deployed. Ensure `AUTH_SERVICE_URL` in that Lambda's environment points to the auth Lambda Function URL from Step 3.

4.8. Step 6 — Build and Deploy Frontend

```
cd frontend/web
npm ci
```

```
npm run build

aws s3 sync dist/ s3://saln-frontend-79bf7afb --delete
aws cloudfront create-invalidation \
    --distribution-id EQEE7KAFB0063 --paths "/*"
```

Or use the helper script from the repo:

```
cd infra/frontend-static
./deploy_frontend.sh
```

4.9. Step 7 — Apply CloudFront Terraform

```
cd infra/frontend-static/terraform
terraform init
# terraform.tfvars already has Lambda domains filled in
terraform apply
```

4.10. Step 8 — CNAME Setup

The CloudFront distribution domain `d1r7w8bsv7kk25.cloudfront.net` must be aliased to `saln1a.upcsweb.dev`. This CNAME DNS record was added by **Prof. Sir Rom Feria** who manages the `upcsweb.dev` domain. Provide him with the CloudFront domain name from `terraform output cloudfront_domain_name`.